JAVA LINKEDLIST

Understanding Linked Lists in Java

1. Learning Objectives

At the end of this lesson, students should be able to:

- Explain what a Linked List is.
- Learn how nodes are structured and connected
- Import java.util.LinkedList.
- Create a Linked List in Java.
- Add elements to a Linked List.
- Access elements.
- Update elements.
- Search for elements.
- Remove elements.
- Display/traverse elements.
- Determine the size of a Linked List.
- Check whether a Linked List is empty.

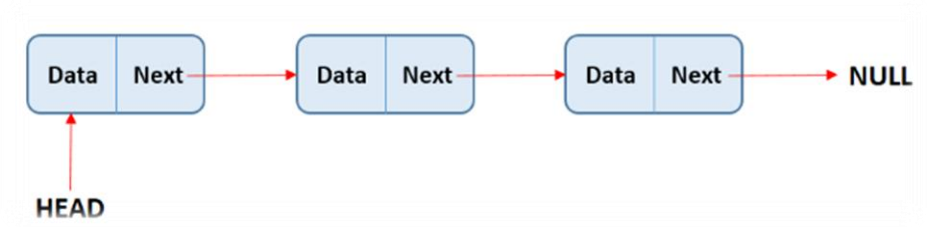
2. What is a Linked List?

A Linked List is a linear data structure where elements (called nodes) are connected using references (links).

Each node contains:

1. Data – the actual value stored in the node.

2. Next – a reference (link) that points to the next node in the list.



Unlike arrays, linked lists do not store elements in contiguous memory locations.
Each node is created dynamically and linked together.

A **Linked List** is a data structure used to store a collection of elements.

For example:

10 → 20 → 30 → 40

We can think of the elements as being connected together.

In Java, we don't have to manually create the Nodes when we use the built-in LinkedList class.

Java provides:

import java.util.LinkedList;

This allows us to use a ready-made Linked List.

3. Importing LinkedList

Before using Java's Linked List, we write:

import java.util.LinkedList;

What does this mean?

It tells Java: "I want to use the LinkedList class that is already provided by Java."

The general format is:

import java.util.LinkedList;

Then we can create a Linked List.

4. Creating a Linked List

The basic syntax is:

```
LinkedList<Integer> numbers = new LinkedList<>();
```

Let's break this down.

LinkedList

This tells Java that we are creating a Linked List.

<Integer>

This tells Java that our Linked List will store **integer values**.

For example:

10

20

30

40

numbers

This is the name of our Linked List.

new LinkedList<>()

This creates the actual Linked List.

5. First Simple Program

```
import java.util.LinkedList;
public class Main {
   public static void main(String[] args) {

      LinkedList<Integer> numbers = new LinkedList<>();

      numbers.add(10);
      numbers.add(20);
      numbers.add(30);

      System.out.println(numbers);
   }
}
```

Output

[10, 20, 30]

6. Step-by-Step Explanation

Step 1 - Import LinkedList

```
import java.util.LinkedList;
```

This allows us to use Java's built-in LinkedList.

Step 2 - Create the Linked List

```
LinkedList<Integer> numbers = new LinkedList<>();
```

We created a Linked List called:

```
numbers
```

It can store integers.

At this point:

```
numbers = [ ]
```

The list is empty.

Step 3 - Add 10

```
numbers.add(10);
```

Now:

```
numbers = [10]
```

Step 4 - Add 20

```
numbers.add(20);
```

Now:

```
numbers = [10, 20]
```

Step 5 - Add 30

```
numbers.add(30);
```

Now:

```
numbers = [10, 20, 30]
```

Step 6 - Display

System.out.println(numbers);

Output:

[10, 20, 30]

Java automatically displays the elements for us.

7. Operation 1 - add()

The add() method is used to add an element.

numbers.add(10);

Example:

```
import java.util.LinkedList;
public class Main {
  public static void main(String[] args) {

    LinkedList<Integer> numbers = new LinkedList<>();
    numbers.add(10);
    numbers.add(20);
    numbers.add(30);
    numbers.add(40);

    System.out.println(numbers);
  }
}
```

Output:

[10, 20, 30, 40]

8. Adding String Values

A Linked List does not have to contain numbers.

We can also store Strings.

```
import java.util.LinkedList;


public class Main {


  public static void main(String[] args) {


    LinkedList<String> names = new LinkedList<>();


    names.add("Ana");
    names.add("Ben");
    names.add("Carlo");


    System.out.println(names);
  }
}
```

Output:

[Anna, Ben, Carlo]

9. LinkedList Data Types

Examples:

Integer

LinkedList<Integer> numbers;

String

LinkedList<String> names;

Double

LinkedList<Double> prices;

Character

LinkedList<Character> letters;

10. Operation 2 - addFirst()

Java provides a method called:

addFirst()

It adds an element at the beginning.

Example:

```
import java.util.LinkedList;
public class Main {
  public static void main(String[] args) {
    LinkedList<Integer> numbers = new LinkedList<>();
    numbers.add(20);
    numbers.add(30);
    numbers.addFirst(10);
    System.out.println(numbers);
  }
}
```

Output:

[10, 20, 30]

11. Operation 3 - addLast()

We can also add an element at the end.

```
numbers.addLast(40);
```

Example:

```
import java.util.LinkedList;

public class Main {

  public static void main(String[] args) {

    LinkedList<Integer> numbers = new LinkedList<>();

    numbers.add(10);
    numbers.add(20);
    numbers.add(30);
    numbers.addLast(40);

    System.out.println(numbers);
  }
}
```

Output:

[10, 20, 30, 40]

12. add() vs addFirst() vs addLast()

Method	Purpose
add()	Add an element
addFirst()	Add at beginning
addLast()	Add at end

Example:

numbers.add(20);

numbers.addFirst(10);

numbers.addLast(30);

Result:

[10, 20, 30]

13. Operation 4 - Insert at a Specific Position

We can also insert an element at a particular index.

Use:

add(index, value)

Example:

numbers.add(1, 20);

Suppose we have:

[10, 30]

Then:

numbers.add(1, 20);

Result:

[10, 20, 30]

14. Understanding the Index

Remember that Java uses **zero-based indexing**.

Value: 10 20 30 40

Index: 0 1 2 3

So:

numbers.add(2, 25);

means:

Insert 25 at index 2.

Result:

[10, 20, 25, 30, 40]

15. Complete Insert Example

```
import java.util.LinkedList;
public class Main {
  public static void main(String[] args) {
    LinkedList<Integer> numbers = new LinkedList<>();
    numbers.add(10);
    numbers.add(20);
    numbers.add(40);
    System.out.println("Before insertion:");
    System.out.println(numbers);
    numbers.add(2, 30);
    System.out.println("After insertion:");
    System.out.println(numbers);
  }
}
```

Output:

Before insertion:

[10, 20, 40]

After insertion:

[10, 20, 30, 40]

16. Operation 5 - Access an Element Using get()

We can retrieve an element using:

get(index)

Example:

```
int value = numbers.get(1);
```

Suppose:

[10, 20, 30, 40]

Indexes:

```
 10   20   30   40
 ↑    ↑    ↑    ↑
 0    1    2    3
```

Therefore:

numbers.get(1)

returns:

20

17. Complete get() Example

```
import java.util.LinkedList;


public class Main {


  public static void main(String[] args) {


    LinkedList<Integer> numbers = new LinkedList<>();


    numbers.add(10);
    numbers.add(20);
    numbers.add(30);


    System.out.println("Element at index 0: " + numbers.get(0));
    System.out.println("Element at index 1: " + numbers.get(1));
    System.out.println("Element at index 2: " + numbers.get(2));
  }
}
```

Output:

```
Element at index 0: 10
Element at index 1: 20
Element at index 2: 30
```

18. Operation 6 - Update an Element Using set()

Suppose we have:

[10, 20, 30]

and we want to change 20 to 25.

Use:

numbers.set(1, 25);

Example:

```
import java.util.LinkedList;
public class Main {
  public static void main(String[] args) {

    LinkedList<Integer> numbers = new LinkedList<>();

    numbers.add(10);
    numbers.add(20);
    numbers.add(30);

    System.out.println("Before:");
    System.out.println(numbers);

    numbers.set(1, 25);

    System.out.println("After:");
    System.out.println(numbers);
  }
}
```

Output:

Before:

[10, 20, 30]

After:

[10, 25, 30]

Remember:

set(index, value)

means:

Replace the value at this index.

19. Operation 7 - Remove an Element by Index

Use:

remove(index)

Suppose:

[10, 20, 30, 40]

We want to remove 30.

Its index is:

2

So:

numbers.remove(2);

Result:

[10, 20, 40]

20. Complete Remove-by-Index Example

```
import java.util.LinkedList;

public class Main {
   public static void main(String[] args) {
      LinkedList<Integer> numbers = new LinkedList<>();


      numbers.add(10);
      numbers.add(20);
      numbers.add(30);
      numbers.add(40);


      System.out.println("Before deletion:");
      System.out.println(numbers);


      numbers.remove(2);


      System.out.println("After deletion:");
      System.out.println(numbers);
   }
}
```

Output:

```
Before deletion:
[10, 20, 30, 40]

After deletion:
[10, 20, 40]
```